

# 2.2 Programming fundamentals

Where this unit sits, what it covers, and how far

## Where this unit sits

|                                    |                                                                                                                                                                                                                                              |
|------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Qualification                      | GCSE Computer Science (9-1), specification code J277, from 2020                                                                                                                                                                              |
| Component                          | Component 02: Computational thinking, algorithms and programming — 80 marks, 1 hour 30 minutes                                                                                                                                               |
| What OCR says the component covers | computational thinking: algorithms, programming techniques, producing robust programs, computational logic and translators                                                                                                                   |
| This unit                          | topic 2.2 Programming fundamentals                                                                                                                                                                                                           |
| Checked                            | 3 October 2026, at <a href="https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/">https://www.ocr.org.uk/qualifications/gcse/computer-science-j277-from-2020/specification-at-a-glance/</a> |

**How far this has been checked.** The topic numbers and titles below are the ones this course uses on its worksheets and slides. They were not re-checked against the published specification document when this map was built: the document could not be retrieved from this build environment. Check them against <https://www.ocr.org.uk/Images/558027-specification-gcse-computer-science-j277.pdf> before relying on this map for a scheme of work, and treat any finer subdivision as absent rather than assumed - no sub-point numbers are claimed anywhere in these documents, because none were verified.

**Before it:** 1.1 Systems architecture, 1.2 Memory and storage, 1.3 Computer networks, connections and protocols, 1.4 Network security, 1.5 Systems software, 1.6 Ethical, legal, cultural and environmental impacts of digital technology, 2.1 Algorithms

**After it:** 2.3 Producing robust programs, 2.4 Boolean logic, 2.5 Programming languages and Integrated Development Environments, PY Python programming

## The lesson sequence and the questions it answers

| # | LESSON                             | THE BIG QUESTION                                                                  | ACTIVITIES | MARKS |
|---|------------------------------------|-----------------------------------------------------------------------------------|------------|-------|
| 1 | <b>Variable store</b><br>(pf-101)  | How does a program store data, and how does it decide which line to run next?     | 14         | 21    |
| 2 | <b>Type foundry</b><br>(pf-102)    | Why does a program need to know what type of data it is working with?             | 15         | 25    |
| 3 | <b>Filing room</b><br>(pf-103)     | How does a program keep hold of data after it has stopped running?                | 14         | 18    |
| 4 | <b>Record vault</b><br>(pf-104)    | How does a program store a set of related data, and how does it search it?        | 15         | 23    |
| 5 | <b>Array yard</b><br>(pf-105)      | How does one name hold many values, and how is code broken into reusable pieces?  | 14         | 19    |
| 6 | <b>Chance machine</b><br>(pf-106)  | How does a program produce a number that nobody can predict?                      | 14         | 19    |
| 7 | <b>Revision HQ 2.2</b><br>(pf-107) | Which parts of 2.2 am I secure on, and which do I need to revise before the test? | 15         | 20    |

| # | LESSON                                  | THE BIG QUESTION        | ACTIVITIES | MARKS |
|---|-----------------------------------------|-------------------------|------------|-------|
| 8 | <b>End-of-topic test</b><br>pf-l08-test | assesses the whole unit | 0          | 0     |

## Prior knowledge, vocabulary and skills

**Assumed before the unit:** the outcomes of 1.1, 1.2, 1.3, 1.4, 1.5, 1.6, 2.1

**Vocabulary introduced:** Assignment, Count-controlled iteration, Casting, Concatenation, File handling, Append, Record, Wildcard, Zero-indexing, Sub-program, Pseudo-random, Seed, Iteration

**What a pupil does, not only reads:** calc, match, mcq, multi, order, sort

## Coverage matrix

Every outcome this unit states, and whether it is taught, practised and assessed **within this unit**. Full means all three; partial means some; not covered means none of the three could be traced.

**How this was worked out.** The trace is derived by matching the content words of an outcome against the station text, the activities and the exam questions of the unit, not authored by hand. The words each match was made on are in `build/coverage.json`. It shows where to look; it does not certify that the alignment is right, and a full row has not been checked by a person either.

| OUTCOME                                                                                         | LESSON | TAUGHT | PRACTISED | ASSESSED | COVERAGE |
|-------------------------------------------------------------------------------------------------|--------|--------|-----------|----------|----------|
| know what variables and constants are, and how a value is assigned to each<br>pf-l01-o1         | pf-l01 | yes    | yes       | yes      | Full     |
| understand how input and output move data in and out of a program<br>pf-l01-o2                  | pf-l01 | yes    | yes       | yes      | Full     |
| know what sequence, selection and iteration each do to the order a program runs in<br>pf-l01-o3 | pf-l01 | yes    | yes       | yes      | Full     |
| know the data types integer, real, Boolean, character and string<br>pf-l02-o1                   | pf-l02 | yes    | yes       | yes      | Full     |
| understand why casting is needed and how a value is cast from one type to another<br>pf-l02-o2  | pf-l02 | yes    | yes       | yes      | Full     |
| know how the arithmetic, comparison and Boolean operators are used<br>pf-l02-o3                 | pf-l02 | yes    | yes       | yes      | Full     |
| know why a program needs to store data in a file<br>pf-l03-o1                                   | pf-l03 | yes    | yes       | yes      | Full     |
| understand how a file is opened, read from, written to and closed<br>pf-l03-o2                  | pf-l03 | yes    | yes       | yes      | Full     |
| know the difference between writing to a file and appending to it<br>pf-l03-o3                  | pf-l03 | yes    | yes       | yes      | Full     |
| know what is meant by a record, a field and a table of records<br>pf-l04-o1                     | pf-l04 | yes    | yes       | yes      | Full     |

| OUTCOME                                                                                                                     | LESSON | TAUGHT | PRACTISED | ASSESSED | COVERAGE    |
|-----------------------------------------------------------------------------------------------------------------------------|--------|--------|-----------|----------|-------------|
| understand how SELECT, FROM and WHERE are used to search for data<br><a href="#">pf-l04-o2</a>                              | pf-l04 | yes    | yes       | yes      | Full        |
| know how AND, OR, NOT and LIKE narrow a search down<br><a href="#">pf-l04-o3</a>                                            | pf-l04 | yes    | yes       | yes      | Full        |
| know how one- and two-dimensional arrays are declared, read and changed<br><a href="#">pf-l05-o1</a>                        | pf-l05 | yes    | yes       | yes      | Full        |
| understand why arrays are zero-indexed and how a loop iterates over one<br><a href="#">pf-l05-o2</a>                        | pf-l05 | yes    | yes       | yes      | Full        |
| know the difference between a procedure and a function, and between local and global variables<br><a href="#">pf-l05-o3</a> | pf-l05 | yes    | yes       | yes      | Full        |
| know what random numbers are used for in a program<br><a href="#">pf-l06-o1</a>                                             | pf-l06 | yes    | yes       | yes      | Full        |
| understand how a random whole number is generated within a given range<br><a href="#">pf-l06-o2</a>                         | pf-l06 | yes    | yes       | yes      | Full        |
| know that a computer produces pseudo-random numbers from an algorithm and a seed<br><a href="#">pf-l06-o3</a>               | pf-l06 | yes    | yes       | yes      | Full        |
| know the whole of topic 2.2 well enough to answer without notes<br><a href="#">pf-l07-o1</a>                                | pf-l07 | —      | —         | —        | Not covered |
| understand the mistakes that most often lose marks in programming questions<br><a href="#">pf-l07-o2</a>                    | pf-l07 | —      | —         | —        | Not covered |
| know which lessons are secure and which still need revision before the test<br><a href="#">pf-l07-o3</a>                    | pf-l07 | —      | —         | —        | Not covered |

**18 full** · **0 partial** · **3 not covered**, of 21 outcomes.

## Gaps, assumptions and overlap

None found by the trace.

Revise 360 — a product of Revise 360 Ltd, created by Olly Pettitt. An independent resource: not written, endorsed or approved by OCR or any exam board. Classroom use and personal study only.